國立臺北科技大學
National Taipei University of Technology

# 資料結構

# Data Structure

# Lab 08

姓名： 紀宥丞

學號： 114310221

| Lab08-Ex1. |
| --- |
| 將範例中的 Inorder traversal function 修改為 Postorder traversal |
| Code |

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 樹的節點
class TreeNode {
public:
    int value;              // 節點的值
    TreeNode* left;         // 左子節點
    TreeNode* right;        // 右子節點
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
};

// 樹結構
class BinaryTree {
public:
    TreeNode* root;
    BinaryTree() : root(nullptr) {} // 初始化樹

    // 用陣列構建二元樹
    TreeNode* buildTree(vector<int>& arr) {
        if (arr.empty()) return nullptr;

        queue<TreeNode*> q; // 儲存待處理的節點
        root = new TreeNode(arr[0]); // 建立根節點(陣列第一個元素)
        q.push(root); // 將根節點加入 queue

        size_t i = 1; // 陣列索引
        while (!q.empty()) {
            TreeNode* current = q.front(); // 取出 queue 中的節點
            q.pop();
```

```
            // 添加左子節點
            if (i < arr.size()) {
                current->left = new TreeNode(arr[i]);
                q.push(current->left); // 將左子節點加入 queue
                i++;
            }
            // 添加右子節點
            if (i < arr.size()) {
                current->right = new TreeNode(arr[i]);
                q.push(current->right); // 將右子節點加入 queue
                i++;
            }
        }
        return root;
    }

    // 中序遍歷
    void inorderTraversal(TreeNode* node) {
        if (node == nullptr) return; //如果節點為空,忽略

        inorderTraversal(node->left);   //遍歷左子樹
        cout << node->value << " ";      // 訪問當前節點
        inorderTraversal(node->right); //遍歷右子樹
    }

    // 後序遍歷
    void postorderTraversal(TreeNode* node) {
        if (node == nullptr) return; //如果節點為空,忽略

        postorderTraversal(node->left);   // 1. 遍歷左子樹
        postorderTraversal(node->right); // 2. 遍歷右子樹
        cout << node->value << " ";          // 3. 訪問當前節點
    }
};

int main() {
    BinaryTree tree; //宣告二元樹 tree
```

```
    // 輸入陣列建立樹
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };
    tree.buildTree(arr); //建立樹

    // 中序遍歷輸出
    cout << "Inorder Traversal: ";
    tree.inorderTraversal(tree.root);
    cout << endl;

    // 後序遍歷輸出
    cout << "Postorder Traversal: ";
    tree.postorderTraversal(tree.root);
    cout << endl;

    return 0;
}
```

**Lab08-Ex2.**

**找出子樹中的最大值:**

**Code**

```
#include <iostream>
#include <queue>
#include <vector>
#include <climits> // 引入 climits 以使用 INT_MIN 作為初始比較值

using namespace std;

// 樹的節點
class TreeNode {
public:
    int value;              // 節點的值
    TreeNode* left;         // 左子節點
    TreeNode* right;        // 右子節點
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
};

// 樹結構
class BinaryTree {
```

```
public:
    TreeNode* root;
    BinaryTree() : root(nullptr) {} // 初始化樹

    // 用陣列構建二元樹
    TreeNode* buildTree(vector<int>& arr) {
        if (arr.empty()) return nullptr;

        queue<TreeNode*> q; // 儲存待處理的節點
        root = new TreeNode(arr[0]); // 建立根節點(陣列第一個元素)
        q.push(root); // 將根節點加入 queue

        size_t i = 1; // 陣列索引
        while (!q.empty()) {
            TreeNode* current = q.front(); // 取出 queue 中的節點
            q.pop();

            // 添加左子節點
            if (i < arr.size()) {
                current->left = new TreeNode(arr[i]);
                q.push(current->left); // 將左子節點加入 queue
                i++;
            }
            // 添加右子節點
            if (i < arr.size()) {
                current->right = new TreeNode(arr[i]);
                q.push(current->right); // 將右子節點加入 queue
                i++;
            }
        }
        return root;
    }

    // 中序遍歷
    void inorderTraversal(TreeNode* node) {
        if (node == nullptr) return; //如果節點為空,忽略

        inorderTraversal(node->left);   //遍歷左子樹
```

```
        cout << node->value << " ";      // 訪問當前節點
        inorderTraversal(node->right); //遍歷右子樹
    }

    // 尋找子樹中的最大值
    int findMax(TreeNode* node) {
        if (node == nullptr) {
            return INT_MIN; // 如果節點為空，回傳極小值，避免影響最大值判斷
        }

        int currentVal = node->value;                // 取得當前節點的值
        int leftMax = findMax(node->left);          // 遞迴找出左子樹的最大值
        int rightMax = findMax(node->right);       // 遞迴找出右子樹的最大值

        // 比較並回傳三者之中的最大值
        int maxVal = currentVal;
        if (leftMax > maxVal) maxVal = leftMax;
        if (rightMax > maxVal) maxVal = rightMax;

        return maxVal;
    }
};

int main() {
    BinaryTree tree; //宣告二元樹 tree

    // 輸入陣列建立樹
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };
    tree.buildTree(arr); //建立樹

    // 中序遍歷輸出
    cout << "Inorder Traversal: ";
    tree.inorderTraversal(tree.root);
    cout << endl;

    // 分別在左右子樹中找出最大值
    if (tree.root != nullptr) {
        int maxLeft = tree.findMax(tree.root->left);     // 傳入左子節點
```

```
        int maxRight = tree.findMax(tree.root->right); // 傳入右子節點

        cout << "Max left subtree value: " << maxLeft << endl;
        cout << "Max right subtree value: " << maxRight << endl;
    }

    return 0;
}
```

**Discussion Section**

說明如何在左右子樹中找到最大值:

使用遞迴（Recursion）。撰寫一個 findMax 函式，傳入子樹的根節點。函式會比較「當前節點的值」、「左子樹遞迴找出的最大值」以及「右子樹遞迴找出的最大值」，並回傳這三個數字中最大的那個。只要在主程式中，分別將 tree.root->left 和 tree.root->right 當作參數傳入 findMax 函式，就能分別得到左子樹與右子樹的最大值。